

Este proyecto simula el software que usaría un técnico de mantenimiento en una torre eléctrica, un bombero o un guardia de seguridad. Dado que el emulador de Android Studio tiene un panel dedicado para **simular fuerzas físicas (Aceleración y Rotación)** moviendo un render 3D del reloj con el mouse, podrán experimentar cómo los cambios bruscos en las fuerzas moleculares de los sensores activan hilos de seguridad.

Tutorial: Sistema de Telemetría de Movimiento y Alerta de Incidentes en Wear OS

Este proyecto práctico enseña a consumir los sensores de movimiento del dispositivo a través de flujos de datos continuos (*Streams*), aplicando lógica de umbrales matemáticos para detectar anomalías físicas y disparar alertas de red asíncronas.

1. Fundamentos Teóricos y Conceptos del Proyecto

El desarrollo de software orientado a la seguridad industrial con *wearables* exige comprender el procesamiento de señales de hardware en tiempo real:

- **Vectores de Aceleración (*G*-Force):** El acelerómetro mide la fuerza de gravedad y el movimiento en tres ejes (X, Y, Z). En estado de reposo plano, la suma vectorial de las fuerzas es aproximadamente $9.81m/s^2$ (o $1G$). Una caída libre altera drásticamente este vector a casi cero, seguido de un pico masivo al impactar contra el suelo.
 - **Gestión de Hilos Continuos (*StreamBuilder*):** A diferencia de una petición HTTP única, los sensores de movimiento transmiten datos docenas de veces por segundo. Escuchar estos datos consume memoria; por ello aprenderán a abrir un canal reactivo (*Stream*) y a cerrarlo de forma mandatoria (*dispose*) para evitar fugas de memoria en el reloj.
 - **Flujo de Confirmación con Cuenta Regresiva:** Si el reloj detecta un impacto, no debe enviar la alerta de inmediato (podría ser un falso positivo, como un aplauso fuerte). El software debe abrir una ventana de tiempo flotante con un temporizador activo para que el usuario cancele el reporte si se encuentra a salvo.
-

2. Configuración del Entorno (Plataforma Nativa)

El proyecto requiere acceso a internet para enviar el reporte de incidentes simulado a un webhook y la declaración de permisos base.

En Android (`android/app/src/main/AndroidManifest.xml`)

Asegúrate de tener el permiso de internet activo en tu manifiesto nativo (como en la práctica anterior):

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-feature android:name="android.hardware.type.watch" />
```

📁 Estructura de Carpetas del Módulo de Telemetría

Dentro de nuestro proyecto dedicado para el reloj, agregamos la nueva vista dentro de la carpeta de pantallas:

```
mi_proyecto_wearable/  
├── pubspec.yaml  
└── lib/  
    ├── main.dart  
    ├── app.dart  
    └── screens/  
        └── watch_telemetry_screen.dart <-- Pantalla del detector de impacto
```

Dependencias del `pubspec.yaml`

Para capturar los sensores de movimiento de forma unificada, añadiremos el paquete oficial de sensores mantenido por el equipo de Flutter, junto con `http`:

```
dependencies:  
  flutter:  
    sdk: flutter  
  wear_plus: ^2.1.1  
  http: ^1.2.0  
  sensors_plus: ^6.1.1 # Biblioteca estándar para acelerómetro y giroscopio
```

3. Código Fuente Completo y Comentado

Archivo: `lib/screens/watch_telemetry_screen.dart`

```
import 'package:flutter/material.dart';  
import 'package:wear_plus/wear_plus.dart';  
import 'package:sensors_plus/sensors_plus.dart';  
import 'package:http/http.dart' as http;  
import 'dart:async';  
import 'dart:math';  
  
class WatchTelemetryScreen extends StatefulWidget {  
  const WatchTelemetryScreen({super.key});  
  
  @override  
  State<WatchTelemetryScreen> createState() => _WatchTelemetryScreenState();  
}  
  
class _WatchTelemetryScreenState extends State<WatchTelemetryScreen> {
```

```

// Suscriptor al flujo de datos del hardware
StreamSubscription<AccelerometerEvent>? _accelerometerSubscription;

// Variables de control de telemetría
double _magnitudo = 0.0;
bool _incidentTriggered = false;
int _countdown = 10;
Timer? _countdownTimer;
String _serverStatus = "Monitoreando operador...";

@override
void initState() {
  super.initState();
  _startTelemetryStream();
}

// 1. Apertura del canal reactivo del Acelerómetro
void _startTelemetryStream() {
  _accelerometerSubscription =
    accelerometerEventStream().listen((AccelerometerEvent event) {
      if (!mounted || _incidentTriggered) return;

      setState(() {
        // Calculamos la magnitud del vector tridimensional: Teorema de Pitágoras
        3D
        // Magnitud = sqrt(x² + y² + z²)
        _magnitudo = sqrt(pow(event.x, 2) + pow(event.y, 2) + pow(event.z, 2));

        // CRITERIO DE INGENIERÍA (Umbral de Impacto):
        // Si la magnitud supera los 25 m/s² (Aproximadamente un impacto de 2.5G),
        // se asume una caída o colisión del operador.
        if (_magnitudo > 25.0) {
          _triggerIncidentProtocol();
        }
      });
    });
}

// 2. Orquestador de Alerta y Cuenta Regresiva de Seguridad
void _triggerIncidentProtocol() {
  setState(() {
    _incidentTriggered = true;
    _countdown = 10;
    _serverStatus = "🚨 ¡IMPACTO DETECTADO! 🚨";
  });

  // Iniciamos un temporizador de un segundo por ciclo
  _countdownTimer = Timer.periodic(const Duration(seconds: 1), (timer) {
    if (!mounted) return;

    setState(() {
      if (_countdown > 1) {
        _countdown--;
      } else {

```

```

        _countdown--;
        _countdownTimer?.cancel();
        _sendEmergencyAlert(); // Si el tiempo se agota, se envía el reporte
    }
    });
});
}

```

// 3. Petición HTTP asíncrona enviando el paquete de telemetría a la nube

```

Future<void> _sendEmergencyAlert() async {
    setState(() { _serverStatus = "Transmitiendo SOS por red..."; });

    try {
        // Simulación de envío a un endpoint de control centralizado
        final url = Uri.parse('https://httpbin.org/post');
        final response = await http.post(url, body: {
            'device': 'WearOS_Sensor_Node',
            'alert_type': 'FALL_DETECTION',
            'force_recorded': _magnitude.toStringAsFixed(2),
            'timestamp': DateTime.now().toIso8601String(),
        });

        if (response.statusCode == 200) {
            setState(() { _serverStatus = "Reporte SOS Enviado Exitosamente."; });
        }
    } catch (e) {
        setState(() { _serverStatus = "Fallo de red en SOS. Reintentando..."; });
    }
}

```

// Cancelación manual del operador (Falso positivo)

```

void _dismissIncident() {
    _countdownTimer?.cancel();
    setState(() {
        _incidentTriggered = false;
        _serverStatus = "Monitoreando operador...";
    });
}

```

@override

```

void dispose() {
    // BUENA PRÁCTICA CRÍTICA: Apagar el sensor para evitar drenado de batería
    _accelerometerSubscription?.cancel();
    _countdownTimer?.cancel();
    super.dispose();
}

```

@override

```

Widget build(BuildContext context) {
    return WatchShape(
        builder: (context, shape, child) {
            return AmbientMode(
                builder: (context, mode, child) {
                    final bool isAmbient = mode == WearMode.ambient;

```

```

final bool isRound = shape == WearShape.round;

return Scaffold(
  backgroundColor: Colors.black,
  body: Center(
    child: Padding(
      padding: EdgeInsets.all(isRound ? 20.0 : 8.0),
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          // --- UI ESTADO NORMAL ---
          if (!_incidentTriggered) ...[
            Icon(
              Icons.security,
              color: isAmbient ? Colors.grey : Colors.greenAccent,
              size: 32,
            ),
            const SizedBox(height: 6),
            Text(
              _serverStatus,
              style: const TextStyle(fontSize: 11, fontWeight:
FontWeight.bold),
              textAlign: TextAlign.center,
            ),
            const SizedBox(height: 10),
            // Consola de telemetría viva en modo activo
            if (!isAmbient)
              Text(
                "Fuerza Actual:\n${_magnitude.toStringAsFixed(2)}
m/s²",
                style: const TextStyle(fontFamily: 'monospace',
fontSize: 12, color: Colors.cyan),
                textAlign: TextAlign.center,
              ),
          ],

          // --- UI ESTADO DE ALERTA CRÍTICA ---
          if (_incidentTriggered) ...[
            Icon(Icons.warning, color: isAmbient ? Colors.grey :
Colors.redAccent, size: 36),
            const SizedBox(height: 4),
            Text(
              _serverStatus,
              style: const TextStyle(fontSize: 10, color:
Colors.redAccent, fontWeight: FontWeight.bold),
              textAlign: TextAlign.center,
            ),
            const SizedBox(height: 6),

            if (_countdown > 0) ...[
              Text(
                "Reporte automático en: $_countdown",
                style: const TextStyle(fontSize: 12, fontWeight:
FontWeight.bold),

```

```

        textAlign: TextAlign.center,
      ),
      const SizedBox(height: 8),
      // Botón de cancelación rápida (No disponible en Modo
Ambiente por seguridad)
      if (!isAmbient)
        SizedBox(
          height: 28,
          child: ElevatedButton(
            onPressed: _dismissIncident,
            style: ElevatedButton.styleFrom(
              backgroundColor: Colors.white24,
              foregroundColor: Colors.white,
              shape: const StadiumBorder(),
            ),
            child: const Text('ESTOY BIEN', style:
TextStyle(fontSize: 10)),
          ),
        ),
      ] else ...[
        // Mensaje final tras el envío del SOS
        Text(_serverStatus, style: const TextStyle(fontSize:
11), textAlign: TextAlign.center),
      ]
    ],
  ],
),
),
),
);
},
);
},
);
}
}

```

Recuerda modificar tu archivo `Lib/app.dart` para que apunte a `home: const WatchTelemetryScreen()`.

4. Guía para Simular el Hardware en el Emulador

Dado que el emulador está fijo en la pantalla simularán la gravedad usando el panel de sensores embebido:

1. Inicia la aplicación en el emulador de Wear OS. Verás el indicador en verde marcando una fuerza de $\sim 9.81 \text{ m/s}^2$ (la gravedad de la Tierra en reposo).
2. Abre los controles extendidos del emulador (los tres puntos `...` de la barra lateral).
3. Ve a la pestaña **Virtual Sensors** (Sensores Virtuales) y selecciona la sección **Device Pose** (Postura del Dispositivo).
4. **Simular la caída:** Verás un render 3D del reloj. Toma cualquiera de los deslizadores de rotación (X , Y , Z) o mueve el reloj de forma brusca y rápida con el puntero del mouse.

5. Al agitar el modelo 3D, el acelerómetro virtual inyectará picos de fuerza. En cuanto la gráfica supere los $25.0m/s^2$, la interfaz de tu app en Wear OS cambiará instantáneamente al color rojo de alerta y comenzará la cuenta regresiva de 10 segundos en tiempo real.